



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Lechuga Castillo Shareny Ixchel

N° de Cuenta: 319004252

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 04

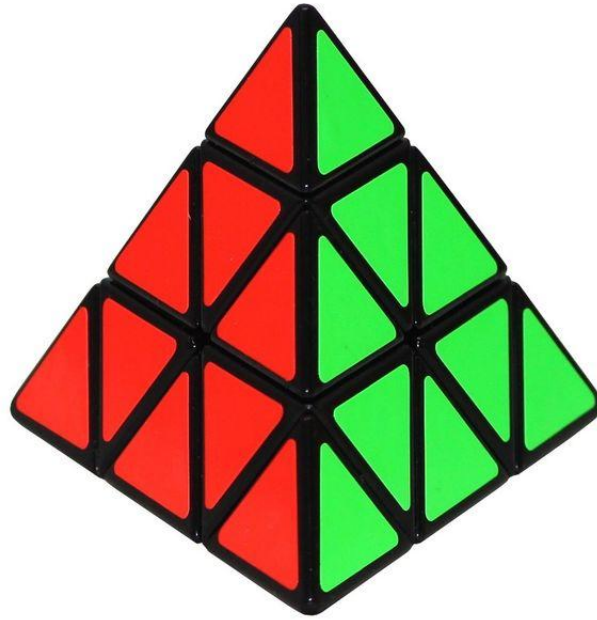
SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 11-09-2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Generar una pirámide rubik (pyraminx) de 9 **pirámides** por cara. Cada cara de la pyraminx que se vea de un color diferente y que se vean las separaciones entre instancias (las líneas oscuras son las que permiten diferenciar cada pirámide pequeña) Agregar en su documento escrito las capturas de pantalla necesarias para que se vean las 4 caras de toda la pyraminx o un video en el cual muestra las 4 caras



Código:

```
42 void CrearPiramideTriangular()
43 {
44     //----- BASE PIRAMIDE (NEGRA) -----
45     unsigned int indices_piramide_triangular[] = {
46         0, 1, 2,
47         1, 3, 2,
48         3, 0, 2,
49         1, 0, 3
50     };
51
52     float max = 2.0f;
53
54     GLfloat vertices_piramide_triangular[] = {
55         -0.65f * max, -0.55f * max, 0.0f * max,
56         0.65f * max, -0.55f * max, 0.0f * max,
57         0.0f * max, 0.7f * max, -0.36f * max,
58         -0.1f * max, -0.57f * max, -1.1f * max
59     };
60
61     Mesh* obj1 = new Mesh();
62     obj1->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 12, 12);
63     meshList.push_back(obj1);
64
65     //----- PIRAMIDES -----
66     unsigned int punta_indices_piramide_triangular[] = {
67         0, 1, 2,
68         1, 3, 2,
69         3, 0, 2,
70         1, 0, 3
71     };
72
73     GLfloat punta_vertices_piramide_triangular[] = {
74         -0.7f, -0.7f, 0.0f, // Vertice 0
75         0.7f, -0.7f, 0.0f, // Vertice 1
76         0.0f, 0.7f, -0.3f, // Vertice 2 Cima
77         0.0f, -0.65f, -1.0f // Vertice 3 Base de atras
78     };
79 }
```

```

80     Mesh* punta = new Mesh();
81     punta->CreateMesh(punta_vertices_piramide_triangular, punta_indices_piramide_triangular, 12, 12);
82     meshList.push_back(punta);
83 }
84
85 void CreateShaders()
86 {
87     Shader *shader1 = new Shader();
88     shader1->CreateFromFiles(vShader, fShader);
89     shaderList.push_back(*shader1);
90
91     Shader* shader2 = new Shader();
92     shader2->CreateFromFiles(vShaderColor, fShader);
93     shaderList.push_back(*shader2);
94 }
95
96
97 int main()
98 {
99     mainWindow = Window(800, 600);
100     mainWindow.Initialise();
101
102     CrearPiramideTriangular();
103     CreateShaders();
104
105     camera = Camera(glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f), -60.0f, 0.0f, 0.3f, 0.3f);
106
107     GLuint uniformProjection = 0;
108     GLuint uniformModel = 0;
109     GLuint uniformView = 0;
110     GLuint uniformColor = 0;
111     glm::mat4 projection = glm::perspective(glm::radians(60.0f), mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 100.0f);
112
113     while (!mainWindow.getShouldClose())
114     {
115         GLfloat now = glfwGetTime();
116         deltaTime = now - lastTime;
117         deltaTime += (now - lastTime) / limitFPS;
118         lastTime = now;
119         glfwPollEvents();
120         camera.keyControl(mainWindow.getKeys(), deltaTime);
121         camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());
122
123         glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
124         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
125
126         // Renderizar la base (piramide negra)
127         shaderList[0].useShader();
128         uniformModel = shaderList[0].getModelLocation();
129         uniformProjection = shaderList[0].getProjectLocation();
130         uniformView = shaderList[0].getViewLocation();
131         uniformColor = shaderList[0].getColorLocation();
132
133         glm::mat4 model = glm::mat4(1.0);
134         model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
135         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
136         glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
137         glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
138         glm::vec3 color = glm::vec3(0.0f, 0.0f, 0.0f); // Color negro
139         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
140         meshList[0]->RenderMesh(); // Renderizar la piramide negra
141
142         //----- COLOR ROJO -----
143         // Renderizar la punta
144         shaderList[0].useShader();
145         uniformModel = shaderList[0].getModelLocation();
146         uniformProjection = shaderList[0].getProjectLocation();
147         uniformView = shaderList[0].getViewLocation();
148         uniformColor = shaderList[0].getColorLocation();
149
150         model = glm::mat4(1.0);
151         model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.42f));
152         model = glm::rotate(model, glm::radians(-5.0f), glm::vec3(1.0f, 0.0f, 0.0f));
153
154         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
155         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
156         glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
157         glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
158         color = glm::vec3(1.0f, 0.0f, 0.0f);
159         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
160         meshList[1]->RenderMesh();
161
162         // Renderizar segundo piso (1er triangulo de izquierda a derecha)
163         shaderList[0].useShader();
164         uniformModel = shaderList[0].getModelLocation();
165         uniformProjection = shaderList[0].getProjectLocation();
166         uniformView = shaderList[0].getViewLocation();
167         uniformColor = shaderList[0].getColorLocation();
168
169         model = glm::mat4(1.0);
170         model = glm::translate(model, glm::vec3(-0.37f, 0.10f, -4.2f));
171         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
172         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
173         glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
174         glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
175         color = glm::vec3(1.0f, 0.0f, 0.0f);
176         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
177         meshList[1]->RenderMesh();
178
179         // Renderizar segundo piso (enmedio rotado)
180         shaderList[0].useShader();
181         uniformModel = shaderList[0].getModelLocation();
182         uniformProjection = shaderList[0].getProjectLocation();
183         uniformView = shaderList[0].getViewLocation();
184         uniformColor = shaderList[0].getColorLocation();
185
186         model = glm::mat4(1.0);
187         model = glm::translate(model, glm::vec3(0.0f, 0.18f, -4.22f));
188         model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
189         model = glm::rotate(model, glm::radians(25.0f), glm::vec3(1.0f, 0.0f, 0.0f));

```

```

190     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
191     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
192     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
193     glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
194     color = glm::vec3(1.0f, 0.0f, 0.0f);
195     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
196     meshList[1] -> RenderMesh();
197
198     // Renderizar segundo piso (3er triangulo de izquierda a derecha)
199     shaderList[0].useShader();
200     uniformModel = shaderList[0].getModelLocation();
201     uniformProjection = shaderList[0].getProjectLocation();
202     uniformView = shaderList[0].getViewLocation();
203     uniformColor = shaderList[0].getColorLocation();
204
205     model = glm::mat4(1.0);
206     model = glm::translate(model, glm::vec3(0.37f, 0.10f, -4.2f));
207     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
208     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
209     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
210     glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
211     color = glm::vec3(1.0f, 0.0f, 0.0f);
212     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
213     meshList[1] -> RenderMesh();
214
215     // Renderizar tercer piso (1er triangulo de izquierda a derecha)
216     shaderList[0].useShader();
217     uniformModel = shaderList[0].getModelLocation();
218     uniformProjection = shaderList[0].getProjectLocation();
219     uniformView = shaderList[0].getViewLocation();
220     uniformColor = shaderList[0].getColorLocation();
221
222     model = glm::mat4(1.0);
223     model = glm::translate(model, glm::vec3(-0.78f, -0.68f, -4.01f));
224     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
225
226     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
227     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
228     glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
229     color = glm::vec3(1.0f, 0.0f, 0.0f);
230     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
231     meshList[1] -> RenderMesh();
232
233     //Renderizar el tercer piso (1er rotado)
234     shaderList[0].useShader();
235     uniformModel = shaderList[0].getModelLocation();
236     uniformProjection = shaderList[0].getProjectLocation();
237     uniformView = shaderList[0].getViewLocation();
238     uniformColor = shaderList[0].getColorLocation();
239
240     model = glm::mat4(1.0);
241     model = glm::translate(model, glm::vec3(0.39f, -0.6f, -4.04f));
242     model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
243     model = glm::rotate(model, glm::radians(25.0f), glm::vec3(1.0f, 0.0f, 0.0f));
244
245     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
246     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
247     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
248     glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
249     color = glm::vec3(1.0f, 0.0f, 0.0f);
250     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
251     meshList[1] -> RenderMesh();
252
253     // Renderizar tercer piso (3er triangulo de izquierda a derecha)
254     shaderList[0].useShader();
255     uniformModel = shaderList[0].getModelLocation();
256     uniformProjection = shaderList[0].getProjectLocation();
257     uniformView = shaderList[0].getViewLocation();
258     uniformColor = shaderList[0].getColorLocation();
259
260     model = glm::mat4(1.0);
261     model = glm::translate(model, glm::vec3(0.0f, -0.68f, -4.01f));
262     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
263     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
264     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
265     glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
266     color = glm::vec3(1.0f, 0.0f, 0.0f);
267     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
268     meshList[1] -> RenderMesh();
269
270     //Renderizar el tercer piso (2do rotado)
271     shaderList[0].useShader();
272     uniformModel = shaderList[0].getModelLocation();
273     uniformProjection = shaderList[0].getProjectLocation();
274     uniformView = shaderList[0].getViewLocation();
275     uniformColor = shaderList[0].getColorLocation();
276
277     model = glm::mat4(1.0);
278     model = glm::translate(model, glm::vec3(-0.39f, -0.6f, -4.04f));
279     model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
280     model = glm::rotate(model, glm::radians(25.0f), glm::vec3(1.0f, 0.0f, 0.0f));
281
282     model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
283     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
284     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
285     glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
286     color = glm::vec3(1.0f, 0.0f, 0.0f);
287     glUniform3fv(uniformColor, 1, glm::value_ptr(color));
288     meshList[1] -> RenderMesh();
289
290     // Renderizar tercer piso (5to triangulo de izquierda a derecha)
291     shaderList[0].useShader();
292     uniformModel = shaderList[0].getModelLocation();
293     uniformProjection = shaderList[0].getProjectLocation();
294     uniformView = shaderList[0].getViewLocation();

```

```

294         uniformColor = shaderList[0].getColorLocation();
295
296         model = glm::mat4(1.0);
297         model = glm::translate(model, glm::vec3(0.78f, -0.68f, -4.01f));
298         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
299         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
300         glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
301         glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
302         color = glm::vec3(1.0f, 0.0f, 0.0f);
303         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
304         meshList[1] -> RenderMesh();
305
306         //----- COLOR AZUL -----
307
308         // Renderizar la punta
309         shaderList[0].useShader();
310         uniformModel = shaderList[0].getModelLocation();
311         uniformProjection = shaderList[0].getProjectLocation();
312         uniformView = shaderList[0].getViewLocation();
313         uniformColor = shaderList[0].getColorLocation();
314
315         model = glm::mat4(1.0);
316         model = glm::translate(model, glm::vec3(0.032f, 0.9f, -4.88f));
317         model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 0.7f, 0.0f));
318         model = glm::rotate(model, glm::radians(5.0f), glm::vec3(0.0f, 0.0f, 1.0f));
319         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
320         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
321         glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
322         glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
323         color = glm::vec3(0.0f, 0.0f, 1.0f);
324         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
325         meshList[1] -> RenderMesh();
326
327         // Renderizar 2da fila (primero)
328         shaderList[0].useShader();
329         uniformModel = shaderList[0].getModelLocation();
330
331
332         model = glm::mat4(1.0);
333         model = glm::translate(model, glm::vec3(-0.07f, 0.1f, -5.34f));
334         model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 1.0f, 0.0f));
335         model = glm::rotate(model, glm::radians(2.0f), glm::vec3(0.0f, 0.0f, 1.0f));
336         model = glm::rotate(model, glm::radians(0.5f), glm::vec3(1.0f, 0.0f, 0.0f));
337         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
338         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
339         glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
340         glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
341         color = glm::vec3(0.0f, 0.0f, 1.0f);
342         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
343         meshList[1] -> RenderMesh();
344
345         // Renderizar 2da fila (rotado)
346         shaderList[0].useShader();
347         uniformModel = shaderList[0].getModelLocation();
348         uniformProjection = shaderList[0].getProjectLocation();
349         uniformView = shaderList[0].getViewLocation();
350         uniformColor = shaderList[0].getColorLocation();
351
352         model = glm::mat4(1.0);
353         model = glm::translate(model, glm::vec3(-0.3f, 0.1f, -5.05f)); // -0.3 x
354
355         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
356         model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f)); // Rotando
357         model = glm::rotate(model, glm::radians(59.0f), glm::vec3(0.0f, 1.0f, 0.0f));
358         model = glm::rotate(model, glm::radians(42.0f), glm::vec3(0.0f, 0.0f, 1.0f));
359         model = glm::rotate(model, glm::radians(-21.0f), glm::vec3(1.0f, 0.0f, 0.0f));
360         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
361         glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
362         glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
363         color = glm::vec3(0.0f, 0.0f, 1.0f);
364         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
365         meshList[1] -> RenderMesh();
366
367         ---
368
369         // Renderizar 2da fila (segundo)
370         shaderList[0].useShader();
371         uniformModel = shaderList[0].getModelLocation();
372         uniformProjection = shaderList[0].getProjectLocation();
373         uniformView = shaderList[0].getViewLocation();
374         uniformColor = shaderList[0].getColorLocation();
375
376         model = glm::mat4(1.0);
377         model = glm::translate(model, glm::vec3(-0.37f, 0.1f, -4.67f));
378         model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 1.0f, 0.0f));
379         model = glm::rotate(model, glm::radians(2.0f), glm::vec3(0.0f, 0.0f, 1.0f));
380         model = glm::rotate(model, glm::radians(0.5f), glm::vec3(1.0f, 0.0f, 0.0f));
381         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
382         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
383         glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
384         glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
385         color = glm::vec3(0.0f, 0.0f, 1.0f);
386         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
387         meshList[1] -> RenderMesh();
388
389         // Renderizar 3ra fila (primero)
390         shaderList[0].useShader();
391         uniformModel = shaderList[0].getModelLocation();
392         uniformProjection = shaderList[0].getProjectLocation();
393         uniformView = shaderList[0].getViewLocation();
394         uniformColor = shaderList[0].getColorLocation();
395
396         model = glm::mat4(1.0);
397         model = glm::translate(model, glm::vec3(-0.151f, -0.68f, -5.81f));
398         model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 1.0f, 0.0f));
399         model = glm::rotate(model, glm::radians(2.0f), glm::vec3(0.0f, 0.0f, 1.0f));
400         model = glm::rotate(model, glm::radians(0.5f), glm::vec3(1.0f, 0.0f, 0.0f));
401         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
402         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
403         glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
404         glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
405         color = glm::vec3(0.0f, 0.0f, 1.0f);
406         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
407         meshList[1] -> RenderMesh();
408
409         ---

```

```

493 //----- COLOR VERDE -----
494 // Renderizar la punta
495 shaderList[0].useShader();
496 uniformModel = shaderList[0].getModelLocation();
497 uniformProjection = shaderList[0].getProjectLocation();
498 uniformView = shaderList[0].getViewLocation();
499 uniformColor = shaderList[0].getColorLocation();
500
501 model = glm::mat4(1.0);
502 model = glm::translate(model, glm::vec3(0.2f, 0.9f, -4.85f));
503 model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 0.7f, 0.0f));
504 //model = glm::rotate(model, glm::radians(5.0f), glm::vec3(0.0f, 0.0f, 1.0f));
505 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
506 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
507 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
508 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
509 color = glm::vec3(0.0f, 1.0f, 0.0f);
510 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
511 meshList[1] -> RenderMesh();
512
513 // Renderizar la 2a fila (primera)
514 shaderList[0].useShader();
515 uniformModel = shaderList[0].getModelLocation();
516 uniformProjection = shaderList[0].getProjectLocation();
517 uniformView = shaderList[0].getViewLocation();
518 uniformColor = shaderList[0].getColorLocation();
519
520 model = glm::mat4(1.0);
521 model = glm::translate(model, glm::vec3(0.45f, 0.12f, -4.42f));
522 //model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 0.7f, 0.0f));
523 //model = glm::rotate(model, glm::radians(5.0f), glm::vec3(0.0f, 0.0f, 1.0f));
524 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
525 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
526 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
527 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
528 color = glm::vec3(0.0f, 1.0f, 0.0f);
529 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
530
531 //----- COLOR AMARILLO -----
532 // Renderizar esquina de abajo
533 shaderList[0].useShader();
534 uniformModel = shaderList[0].getModelLocation();
535 uniformProjection = shaderList[0].getProjectLocation();
536 uniformView = shaderList[0].getViewLocation();
537 uniformColor = shaderList[0].getColorLocation();
538
539 model = glm::mat4(1.0);
540 model = glm::translate(model, glm::vec3(0.7f, -0.8f, -4.12f));
541 model = glm::rotate(model, glm::radians(-5.0f), glm::vec3(1.0f, 0.0f, 0.0f));
542 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
543 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
544 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
545 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
546 color = glm::vec3(1.0f, 1.0f, 0.0f);
547 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
548 meshList[1] -> RenderMesh();
549
550 // Renderizar 2da esquina de abajo
551 shaderList[0].useShader();
552 uniformModel = shaderList[0].getModelLocation();
553 uniformProjection = shaderList[0].getProjectLocation();
554 uniformView = shaderList[0].getViewLocation();
555 uniformColor = shaderList[0].getColorLocation();
556
557 model = glm::mat4(1.0);
558 model = glm::translate(model, glm::vec3(-0.85f, -0.8f, -4.12f));
559 model = glm::rotate(model, glm::radians(-5.0f), glm::vec3(1.0f, 0.0f, 0.0f));
560 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
561 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
562 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
563 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
564 color = glm::vec3(1.0f, 1.0f, 0.0f);
565 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
566 meshList[1] -> RenderMesh();

```

Problemas:

Ahora si me costo demasiado trabajo esta práctica, desde cómo me sentía de salud hasta enojarme por no poner bien las coordenadas, tuve problemas al rotar e insertar bien cada pirámide, la verdad que el azul ya lo dejé como pude porque ya no aguanté.

3.- Conclusión:

- Los ejercicios del reporte: La complejidad estuvo super elevada a mi parecer, cada vez se pone mucho más difícil esto.

- b. Comentarios generales: Hubiera querido que se hiciera algún ejemplo mucho más específico que solo las esferas, se podía tener o tomar el tiempo de explicar más a detalle cada figura y cada función.
- c. Conclusión: Estuvo muy difícil, no estuve en mi mejor momento para realizar esta práctica pero se logró lo que se pudo lograr.